

Roteiro de Teste da Aplicação — Catálogo de Biblioteca (POO)

Objetivo:

Testar as funcionalidades de **adição, busca, atualização e listagem** de itens da biblioteca (livros, revistas e materiais digitais) e identificar o uso dos **princípios da orientação a objetos**.

Pré-requisitos

1. Ter o **NetBeans** instalado.
 2. Ter criado o projeto com os arquivos Java conforme o modelo fornecido anteriormente.
 3. Executar a classe Main.java (clique com o botão direito sobre ela e selecione "**Run File**").
-

Etapas de Teste

◆ 1. Testar Adição de um Livro

Caminho no menu:

1 - Adicionar Livro

Digite:

- Título: *POO em Java*
- Autor: *João Silva*
- ISBN: *1234567890*

Resultado esperado:

- Mensagem: Item adicionado com sucesso!

Explicação POO:

- A classe Livro herda de Item.
 - O objeto é criado e adicionado à lista no Catalogo.
-

◆ 2. Testar Adição de uma Revista

Menu:

2 - Adicionar Revista

Digite:

- Título: *Revista Ciência*
- Autor: *Maria Souza*
- Edição: *10*

Resultado esperado:

- Mensagem: Item adicionado com sucesso!
-

◆ **3. Testar Adição de Material Digital**

Menu:

3 - Adicionar Material Digital

Digite:

- Título: *Apostila de Algoritmos*
 - Autor: *Carlos Lima*
 - Formato: *PDF*
-

◆ **4. Testar Listagem de Todos os Itens**

Menu:

6 - Listar Itens

Resultado esperado:

- Exibição dos 3 itens adicionados, cada um com seus atributos específicos.
 - O método `exibirDetalhes()` é usado de forma polimórfica.
-

◆ **5. Testar Busca por Autor**

Menu:

4 - Buscar Item

Digite: Maria

Resultado esperado:

- Apenas a Revista de Maria Souza deve ser exibida.
-

◆ **6. Testar Atualização de um Item**

Menu:

5 - Atualizar Item

Digite:

- Título atual: *POO em Java*
- Novo título: *POO Moderna*
- Novo autor: *João S. Oliveira*

Resultado esperado:

- Mensagem: Item atualizado com sucesso!

Depois, execute o menu 6 novamente para confirmar a mudança.

◆ **7. Testar Busca por Título**

Menu:

4 - Buscar Item

Digite: POO Moderna

Resultado esperado:

- Livro com novo título e autor atualizado deve aparecer.
-

◆ **8. Tentar Buscar Item Inexistente**

Menu:

4 - Buscar Item

Digite: Python

Resultado esperado:

- Mensagem: Nenhum item encontrado com o termo: Python
-

◆ **9. Tentar Atualizar Item Inexistente**

Menu:

5 - Atualizar Item

Digite:

- Título atual: *Livro que não existe*
- Novo título: *Teste*
- Novo autor: *Teste*

Resultado esperado:

- Mensagem: Item não encontrado para atualização.

Conceitos de POO reforçados durante os testes

Funcionalidade	Conceito POO	Onde aparece
Adição de itens	Encapsulamento, Herança	Subclasses Livro, Revista, etc
Listagem	Polimorfismo (exibirDetalhes)	Método abstrato sobrescrito nas classes
Atualização	Encapsulamento (setTitulo)	Atualiza atributos de forma segura
Organização	Reuso com DRY	Classe Item centraliza atributos comuns

RESUMO DA APLICAÇÃO BIBLIOTECA:

Item.java (Classe Abstrata Base)

Explicações básicas para quem está começando:

- **Classe abstrata:** é um tipo de classe que não pode ser instanciada diretamente. Serve como modelo para outras classes.
- **Método abstrato:** é um método que não tem corpo aqui. Ele será implementado nas classes filhas (como Livro).
- **Encapsulamento:** os atributos titulo e autor estão protegidos (protected), o que ajuda a proteger os dados e manter o controle de acesso.

Livro.java

Explicações extras para iniciantes:

- **Herança:** a classe Livro herda de Item. Isso significa que Livro já tem os atributos titulo e autor, além dos métodos getTitulo, getAutor, setTitulo, setAutor, etc.
- **super(...):** é usado para chamar o construtor da classe mãe (Item) e evitar repetir código.
- **@Override:** indica que estamos *reescrevendo* um método que já existe na classe pai, mas adaptando o comportamento para Livro.
- **Encapsulamento:** usamos private para proteger o atributo isbn, e criamos métodos get e set para acessá-lo de forma segura.

Revista.java

Explicação para quem está começando:

- **Herança:** Revista herda tudo o que a classe Item possui (título, autor, etc.).
- **Atributo novo:** além dos dados básicos, adicionamos o campo edicao, exclusivo de revistas.
- **Encapsulamento:** usamos métodos get e set para acessar e modificar edicao, protegendo o dado.
- **Construtor com super():** reutiliza o código da classe mãe (Item) para definir o título e autor.
- **@Override:** sobrescreve o método abstrato exibirDetalhes() com uma implementação específica para revistas.

MaterialDigital.java

Explicação para iniciantes:

- **Herança:** a classe MaterialDigital herda atributos e métodos da classe Item, como titulo, autor, getTitulo(), setAutor(), etc.
- **Construtor:** ao criar um novo material digital, usamos o super() para definir os atributos herdados (título e autor) e depois definimos o formato.
- **Encapsulamento:** o atributo formato é private, ou seja, só pode ser acessado usando os métodos getFormato() e setFormato().
- **@Override:** estamos sobrescrevendo o método exibirDetalhes() que foi definido como abstrato na classe Item.

Catalogo.java

Explicações extras para iniciantes:

- **List<Item>:** isso significa que a variável itens pode armazenar vários objetos que herdam de Item (como Livro, Revista, etc).
- **ArrayList<>:** é uma lista flexível, muito usada para armazenar elementos em Java.
- **.toLowerCase():** garante que a busca funcione sem se preocupar com maiúsculas ou minúsculas.
- **.contains():** verifica se um texto está contido em outro, útil para buscas parciais.

Main.java

💡 Para quem está aprendendo:

- **Scanner:** ferramenta que lê o que o usuário digita.
- **Switch-case:** estrutura de decisão para executar um bloco de código dependendo do valor da opção.
- **POO na prática:** você cria *objetos* das classes Livro, Revista, etc., e usa *métodos* para manipulá-los.